# Arquitecturas distribuidas

Cliente-servidor, P2P, layered, SOA, microservicios, event-driven



**Idea central de la sesion**

"Un estilo de arquitectura es una apuesta sobre donde vive el acoplamiento y el fallo, no una lista de nombres."

## Que se trabajo

Arquitectura son **dos preguntas que se responden juntas**: como se parte el sistema (fronteras) y como se comunican las partes (contratos). Regla fija: las buenas fronteras siguen una **capacidad de negocio** (contexto acotado del DDD), nunca una capa tecnica.

## Los seis estilos - apuestas sobre donde vive el acoplamiento

| Estilo | Acoplamiento vive en | Modo de fallo |
|---|---|---|
| Cliente-servidor | Todos dependen de un servidor | Cuello de botella / SPOF |
| Peer-to-peer | La coordinacion misma | Costo de coordinacion y seguridad |
| Por capas / N-tier | Presentacion-logica-datos | Monolito distribuido si se parte mal |
| SOA | El ESB / bus de mensajes | Middleware central pesado |
| Microservicios | La red, entre capacidades | Complejidad operativa |
| Orientado a eventos | El tiempo (productor/consumidor) | Consistencia eventual, dificil de depurar |

**La trampa: el MONOLITO DISTRIBUIDO**

Servicios que **siguen compartiendo la base de datos** y deben desplegarse juntos: se paga el costo de la red y no se suelta nada del acoplamiento.

**Si los servicios no pueden desplegarse independientemente, no son microservicios** - sin importar cuantos repos existan.

## Ejemplo aplicado: "Deberiamos separar el catalogo?"

| Parte | Decision | Razon |
|---|---|---|
| Catalog | **Se extrae** | Contexto acotado real + necesidad de escalado **medida** (picos Black Friday). Almacen propio, optimizado para lectura y cacheado |
| Orders + Payments | **Se quedan juntos** | Acoplados por una transaccion. Se coordinan despues con una Saga, no se parten temprano |

La descomposicion correcta se ve asi: API Gateway al frente de Orders/Payments/Inventory, **cada uno con su propia BD**, hablando solo por contratos (REST/gRPC/eventos). Eso -y no la cantidad de repos- compra despliegue y escalado independientes.

### Errores comunes a evitar

- Partir por **capa tecnica** en vez de capacidad de negocio.
- **Compartir una sola base de datos** entre servicios.
- Meter un bus "porque es empresarial" cuando una llamada directa alcanza.
- Saltarse el **ADR**: despues nadie recuerda por que la arquitectura es como es.

# Arquitecturas distribuidas

Cliente-servidor, P2P, layered, SOA, microservicios, event-driven



**Autoevaluacion de la sesion**

6/6 completado - comparacion de estilos y deteccion del anti-patron de monolito distribuido.

---

*Distributed Systems - Semana 2, Sesion 1 - Faculty of Engineering, CORHUILA 2026-B. Material: code-corhuila.github.io/ova-web/2026-B/distribuidos/*